

歯科医院 架電CRM システム設計書

対象：歯科医院へのテレアポ・悩みヒアリング業務を効率化する、少人数チーム向けの軽量CRM
前提：スマホブラウザでの利用が中心。複数人が同時に架電し、進捗をリアルタイム共有する。

0. 技術選定の結論（Supabase vs Firebase）

Supabase を採用。理由は3点。

1. 本体プロダクトとスタックを揃えられる。リコール自動化SaaSが Next.js + Supabase 前提。認証・RLS・Realtimeのノウハウとコードを使い回せる。
2. リレーショナル+RLSがCRMに最適。「医院 1 : N 架電履歴」という構造をPostgresで素直に表現でき、`auth.users` ベースの行レベル権限制御が標準で効く。
3. Realtimeが標準装備。`postgres_changes` を購読するだけで、要件4「リロードなし即時反映」がほぼ追加実装ゼロで成立する。Firebaseでも可能だが、NoSQLのデータモデリングとセキュリティルールの学習コストがこの規模では割に合わない。

1. システムアーキテクチャ全体像

```
flowchart TD
    subgraph Client["スマホ / PC ブラウザ"]
        UI["Next.js App Router UI"]
    end
    subgraph Vercel["Vercel"]
        SC["Server Components<br/>初期データ取得 (SSR)"]
        MW["Middleware<br/>セッション更新"]
    end
    subgraph Supabase["Supabase"]
        AUTH["Auth<br/>メンバー認証"]
        DB["Postgres<br/>clinics / call_logs / profiles"]
        RT["Realtime<br/>postgres_changes"]
    end

    UI -->|ログイン| AUTH
    UI -->|初回表示| SC
    SC -->|SELECT (RLS適用)| DB
```

UI -->|ステータス/メモ更新| DB
UI <-->|購読：変更を即時受信| RT
RT -. ->|行の変更を配信| DB
MW --- AUTH

処理の流れ

1. メンバーがログイン → Supabase Auth がセッションCookieを発行。
2. 一覧ページ（Server Component）が初期データをSSRで取得 → 初回表示が速い。
3. クライアント側が Realtime チャンネルを購読。
4. 誰かがステータス／メモを更新 → Postgresの行が変更 → 購読中の全クライアントへ即時配信 → リロードなしでUI更新。

設計上のキモ

- **Server Component**で初期取得、**Client Component**で対話・購読を担当する役割分担。
- **RLSはRealtimeにも効く**。クライアントは「自分がSELECTできる行」の変更しか受信しない。本CRMは「認証済みなら全行閲覧可」ポリシーなので、全メンバーに全更新が届く。
- 認証は `@supabase/ssr` でCookieベースに統一（Server/Client/Middlewareで同じセッションを共有）。

2. データベース設計

テーブル一覧

テーブル	役割	関係
profiles	アプリ利用メンバー（auth.users と1:1）	—
clinics	架電対象の歯科医院マスタ	1医院：N履歴
call_logs	架電・ヒアリングの履歴	clinics に従属

設計判断：メモは「単一テキスト欄」ではなく `call_logs`（履歴）として持つ。テレアポは複数回かけるのが前提で、「いつ・誰が・何を聞いたか」を残せないと使い物にならないため。ただし一覧での素早い確認用に、医院側へ直近メモの要約 `latest_memo` を持たせる。

2-1. profiles

カラム	型	制約	説明
id	uuid	PK, FK→auth.users	認証ユーザーと同一ID
display_name	text	NOT NULL	一覧/履歴に表示する名前
created_at	timestampz	NOT NULL, default now()	作成日時

2-2. clinics

カラム	型	制約	説明
id	uuid	PK, default gen_random_uuid()	医院ID
name	text	NOT NULL	医院名
phone	text	—	電話番号（表示・発信用）
address	text	—	住所
prefecture	text	—	都道府県（絞り込み・グループ化用）
city	text	—	市区町村（絞り込み・グループ化用）
business_hours	text	—	診療時間（表示用・Googleからそのまま貼れる）
hours	jsonb	NOT NULL, default '{ }'	構造化した曜日別診療時間（営業中判定・絞り込み用）
status	text	NOT NULL, default 'not_called', CHECK 制約	進捗ステータス（下表）
assigned_to	uuid	FK→profiles, ON DELETE SET NULL	担当者（二重架電防止）
next_action_at	timestampz	—	次回約束・再架電予定日時
latest_memo	text	—	一覧表示用の直近メモ要約
source	text	default 'google'	データ取得元

created_at	timestampz	NOT NULL, default now()	作成日時
updated_at	timestampz	NOT NULL, default now()	更新日時（トリガで自動更新）

ステータス定義（status / outcome 共通）

DB値	画面表示	用途
not_called	未架電	初期値
no_answer	不通	出ない／話中
unavailable	担当者不在	折り返し対象
heard	ヒアリング済	悩みを聞けた
appointment	アポ獲得	ゴール
rejected	お断り	デッドリード

hours（曜日別診療時間）の構造

キーは曜日（mon ～ sun）、値は診療時間帯の配列。昼休みは2区間に分割、空配列＝休診日で表す。

```
{
  "mon": [["09:00", "13:00"], ["14:30", "18:00"]],
  "tue": [["09:00", "13:00"], ["14:30", "18:00"]],
  "wed": [["09:00", "13:00"]],
  "thu": [],
  "fri": [["09:00", "13:00"], ["14:30", "18:00"]],
  "sat": [["09:00", "13:00"]],
  "sun": []
}
```

この形にすると「今かけてよい医院か（営業中／昼休み／休診）」をコードで判定でき、絞り込みにも使える。{}（未登録）の場合は判定せず business_hours の文字列だけ表示する。

2-3. call_logs

カラム	型	制約	説明
-----	---	----	----

id	uuid	PK, default gen_random_uuid()	履歴ID
clinic_id	uuid	NOT NULL, FK→clinics, ON DELETE CASCADE	対象医院
user_id	uuid	NOT NULL, FK→profiles	架電した人
outcome	text	NOT NULL, CHECK制約	この架電の結果ステータス
memo	text	—	聞き出した悩み・次回約束など
created_at	timestampz	NOT NULL, default now()	架電日時

2-4. DDL (Supabase SQL Editor にそのまま貼れる)

-- ===== テーブル =====

```
create table public.profiles (
  id          uuid primary key references auth.users(id) on delete cascade,
  display_name text not null,
  created_at  timestampz not null default now()
);
```

```
create table public.clinics (
  id          uuid primary key default gen_random_uuid(),
  name        text not null,
  phone       text,
  address     text,
  prefecture  text,
  city        text,
  business_hours text,
  hours       jsonb not null default '{} '::jsonb,
  status      text not null default 'not_called'
              check (status in ('not_called','no_answer','unavailable','heard'
assigned_to   uuid references public.profiles(id) on delete set null,
next_action_at timestampz,
latest_memo  text,
source       text default 'google',
created_at   timestampz not null default now(),
updated_at   timestampz not null default now()
);
```

```
create table public.call_logs (
  id          uuid primary key default gen_random_uuid(),
```

```

clinic_id  uuid not null references public.clinics(id) on delete cascade,
user_id    uuid not null references public.profiles(id),
outcome    text not null
           check (outcome in ('not_called','no_answer','unavailable','heard','a
memo        text,
created_at timestamptz not null default now()
);

create index idx_call_logs_clinic on public.call_logs(clinic_id);
create index idx_clinics_status   on public.clinics(status);
create index idx_clinics_region   on public.clinics(prefecture, city);

-- ===== updated_at 自動更新 =====
create or replace function public.set_updated_at()
returns trigger language plpgsql as $$
begin new.updated_at = now(); return new; end; $$;

create trigger trg_clinics_updated_at
  before update on public.clinics
  for each row execute function public.set_updated_at();

-- ===== RLS (少人数の信頼チーム前提：認証済みは全件可) =====
alter table public.profiles  enable row level security;
alter table public.clinics   enable row level security;
alter table public.call_logs enable row level security;

create policy "auth read profiles"   on public.profiles  for select to authentic
create policy "auth manage clinics"  on public.clinics  for all    to authentic
create policy "auth read call_logs"  on public.call_logs for select to authentic
create policy "auth insert call_logs" on public.call_logs for insert to authentic

-- ===== Realtime 配信対象に登録 =====
alter publication supabase_realtime add table public.clinics;
alter publication supabase_realtime add table public.call_logs;

-- UPDATE/DELETE時に変更前の行情報も配信したい場合（任意）
alter table public.clinics  replica identity full;
alter table public.call_logs replica identity full;

```

権限の方針： チームが小さく全員が全医院を見る前提なので「認証済み＝全件可」。将来「担当した医院だけ編集」等に絞る場合は `using (assigned_to = auth.uid())` 等にポリシーを差し替えるだけで対応可。

3. 画面遷移と主要UIコンポーネント設計

画面遷移

```
flowchart LR
    LOGIN["/login<br/>ログイン"] --> LIST["/clinics<br/>医院一覧（メイン）"]
    LIST -->|医院をタップ| DETAIL["/clinics/[id]<br/>医院詳細・履歴"]
    DETAIL -->|戻る| LIST
    LIST -->|電話番号タップ| CALL(("発信<br/>tel:")):::ext
    DETAIL -->|電話番号タップ| CALL
    classDef ext fill:#eef,stroke:#88a;
    class CALL ext;
```

画面ごとの役割

- `/login` ... Supabase Auth でログイン。公開サインアップは無効化し、メンバーはダッシュボードから招待（小規模・セキュア）。
- `/clinics`（メイン） ... 医院カードのリスト。各カードに「医院名・ステータスバッジ・電話発信ボタン・次回予定」。上部に検索＋都道府県／市区町村／ステータスの絞り込み、および「分け方」トグル（都道府県別・市区町村別・ステータス別にセクション分割）。スマホで親指操作しやすいカードUI。
- `/clinics/[id]`（詳細） ... 医院情報フル表示、発信ボタン（大）、ステータス変更、メモ付き架電履歴の追加フォーム、過去履歴のタイムライン、次回予定の設定。

主要コンポーネント

コンポーネント	種別	役割
ClinicListPage	Server	初期データをSSR取得し下へ渡す
ClinicListRealtime	Client	受け取った配列を保持＋Realtime購読＋描画
ClinicCard	Client	1医院の行。名前・ステータスバッジ・診療時間・営業中バッジ・発信・次回予定
StatusBadge	—	進捗ステータスを色付きバッジで表示
OpenStatusBadge	Client	<code>hours</code> から「営業中／昼休み／営業時間外／休診」を判定して表示（JST基準）
StatusSelect	Client	ステータス変更（楽観的更新）
CallButton	—	<code>tel:</code> リンク。タップで発信
CallLogForm	Client	結果＋メモを登録

CallLogTimeline	—	履歴の時系列表示
FilterBar	Client	検索+都道府県+市区町村+ステータス+営業状況（営業中のみ）で絞り込み（市区町村は選択中の都道府県に連動）
GroupToggle	Client	表示の分け方を切替（なし / 都道府県 / 市区町村 / ステータス）
GroupedClinicList	Client	グループ見出し+件数バッジ付きでカードをセクション表示

ワンタップ発信（要件5）

電話番号からハイフン・空白を除去して `tel:` URIにする。

```
// components/call-button.tsx
export function CallButton({ phone }: { phone: string | null }) {
  if (!phone) return null;
  const tel = phone.replace(/[\^d+]/g, ""); // 数字と+以外を除去
  return (
    <a
      href={`tel:${tel}`}
      className="block rounded-xl bg-emerald-600 px-4 py-3 text-center font-bold"
    >
      📞 {phone} に発信
    </a>
  );
}
```

4. 状態管理とリアルタイム同期の実装アプローチ

方針：専用ライブラリは入れない

この規模では Redux/Zustand は不要。「**Server Component**で初期取得 → **Client Component**が `useState` で保持 → **Realtime**で差分更新」で十分。INSERT/UPDATE/DELETE の反映は `useReducer` で整理すると見通しが良い。

Supabaseクライアント（`@supabase/ssr`）

```
// lib/supabase/client.ts （ブラウザ用）
import { createBrowserClient } from "@supabase/ssr";
export function createClient() {
  return createBrowserClient(
```



```

    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  );
}

// lib/supabase/server.ts (Server Component用 / Next.js 15: cookies は await)
import { createServerClient } from "@supabase/ssr";
import { cookies } from "next/headers";
export async function createClient() {
  const store = await cookies();
  return createServerClient(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll: () => store.getAll(),
        setAll: (list) => {
          try { list.forEach(({ name, value, options }) => store.set(name, value, options)); }
        },
      },
    },
  );
}

```

リアルタイム購読フック (中核)

```

// hooks/use-realtime-clinics.ts
"use client";
import { useEffect, useState } from "react";
import { createClient } from "@lib/supabase/client";
import type { Clinic } from "@lib/types";

export function useRealtimeClinics(initial: Clinic[]) {
  const [clinics, setClinics] = useState<Clinic[]>(initial);
  const supabase = createClient();

  useEffect(() => {
    const channel = supabase
      .channel("clinics-changes")
      .on(
        "postgres_changes",
        { event: "*", schema: "public", table: "clinics" },
        (payload) => {
          setClinics((prev) => {
            if (payload.eventType === "INSERT")

```

```

        return [payload.new as Clinic, ...prev];
    if (payload.eventType === "UPDATE")
        return prev.map((c) =>
            c.id === (payload.new as Clinic).id ? (payload.new as Clinic) : c
        );
    if (payload.eventType === "DELETE")
        return prev.filter((c) => c.id !== (payload.old as Clinic).id);
    return prev;
    });
}
)
.subscribe();

return () => { supabase.removeChannel(channel); };
}, []);

return clinics;
}

```

楽観的更新（ステータス変更）

```

async function updateStatus(id: string, status: ClinicStatus) {
    setClinics((prev) => prev.map((c) => (c.id === id ? { ...c, status } : c))); //
    const { error } = await supabase.from("clinics").update({ status }).eq("id", id
    if (error) { /* 失敗時はトースト+元に戻す */ }
}

```

整合性メモ： 更新した本人にも Realtime で同じUPDATEが返ってくるが、id で置換するだけなので二重適用にならない（冪等）。楽観的更新とRealtimeが自然に整合する。

絞り込み・グループ化（派生状態）

Realtimeフックは常に「全件」を返す。絞り込みとグループ分けはサーバーに問い合わせ直さず、クライアント側の派生計算で行う（数百～数千件ならこれで十分速く、切替も瞬時）。

```

// フィルタ状態
type Filters = { q: string; prefecture?: string; city?: string; status?: ClinicSt
type GroupBy = "none" | "prefecture" | "city" | "status";

// 絞り込み → グループ化を useMemo で派生
const filtered = useMemo(() => clinics.filter((c) =>
    (!filters.q || c.name.includes(filters.q)) &&
    (!filters.prefecture || c.prefecture === filters.prefecture) &&
    (!filters.city || c.city === filters.city) &&
    (!filters.status || c.status === filters.status) &&

```

```

    (!filters.onlyOpen || callStatusNow(c.hours) === "open")    // ← 今かけられる医院だ
  ), [clinics, filters]);

const groups = useMemo(() => {
  if (groupBy === "none") return [{ key: "all", label: "すべて", items: filtered }
  const map = new Map<string, Clinic[]>();
  for (const c of filtered) {
    const key = (groupBy === "status" ? STATUS_LABEL[c.status] : c[groupBy]) ?? "
    (map.get(key) ?? map.set(key, []).get(key!)).push(c);
  }
  return [...map.entries()]
    .map(([key, items]) => ({ key, label: key, items }))
    .sort((a, b) => a.label.localeCompare(b.label, "ja"));
}, [filtered, groupBy]);
// → groups を GroupedClinicList に渡し、見出し+件数 (items.length) 付きで描画

```

ポイント：

- 市区町村のプルダウンは選択中の都道府県に連動させる（`clinics` から該当県の `city` を `distinct` 抽出）。県未選択なら全市区町村。
- 都道府県・市区町村の選択肢はマスタ表を持たず、読み込んだ `clinics` から動的に算出（`[...new Set(clinics.map(c => c.prefecture))]`）。データ投入された地域だけが選択肢に出るので運用が楽。

営業中の判定（架電可否）

`hours` から「今かけてよいか」を判定する純粋関数。必ず日本時間（JST）基準で評価する（端末のタイムゾーンに依存させない）。

```

type Interval = [string, string]; // 例 ["09:00", "13:00"]
type WeeklyHours = Partial<Record<"sun"|"mon"|"tue"|"wed"|"thu"|"fri"|"sat", Interval>>
type CallStatus = "open" | "lunch" | "closed_today" | "off" | "unknown";

const DAY_KEYS = ["sun", "mon", "tue", "wed", "thu", "fri", "sat"] as const;

export function callStatusNow(hours: WeeklyHours): CallStatus {
  if (!hours || Object.keys(hours).length === 0) return "unknown"; // 未登録なら判定

  // 現在のJSTの曜日・時刻を取得
  const p = new Intl.DateTimeFormat("en-GB", {
    timeZone: "Asia/Tokyo", weekday: "short", hour: "2-digit", minute: "2-digit",
  }).formatToParts(new Date());
  const wd = p.find((x) => x.type === "weekday")!.value.toLowerCase().slice(0, 3)
  const hhmm = `${p.find((x) => x.type === "hour")!.value}:${p.find((x) => x.type

```

```
const today = hours[wd as keyof WeeklyHours];
if (!today || today.length === 0) return "off"; // 休診日

if (today.some(([open, close]) => hhmm >= open && hhmm < close)) return "open";

// 診療日だが時間外：区間と区間の隙間にいれば昼休み、それ以外は営業時間外
const inLunch = today.some(([open, close], i) => today[i + 1] && hhmm >= close && h
return inLunch ? "lunch" : "closed_today";
}
```

"HH:MM"（ゼロ埋め24時間表記）は文字列比較で時刻の大小がそのまま判定できるので、時刻パースは不要。OpenStatusBadge はこの戻り値を「営業中（緑）／昼休み（黄）／営業時間外（灰）／休診（赤）／時間未登録（無印）」に対応させる。

より厳密に祝日や臨時休診まで扱うなら `date-fns-tz` 等のライブラリ+祝日マスタを足す。MVPは上記で十分。

二重架電の防止（チーム運用上の要点）

複数人が同じ医院に同時にかける事故を防ぐため、

- `assigned_to`（担当者）で医院を分担する、
- ステータスが `Realtime` で即共有されるので「誰かが今 `unavailable` にした」がすぐ見える。

MVPではこの2つで十分実用に足りる。

5. MVP開発ステップ（AIコーディング前提・最短リリース）

各フェーズは「動く状態」を積み上げる順。時間目安はAIツール併用の現実値。

Phase 0 | セットアップ（～半日）

- `pnpm create next-app`（App Router, TypeScript, Tailwind）
- Supabaseプロジェクト作成、`.env.local` に URL / anon key
- `@supabase/supabase-js` `@supabase/ssr` 導入
- 推奨ディレクトリ：

```
app/
  (auth)/login/page.tsx
  (app)/clinics/page.tsx          # 一覧
  (app)/clinics/[id]/page.tsx    # 詳細
```

```
(app)/layout.tsx
lib/supabase/{client,server}.ts
lib/types.ts
components/{clinic-card,status-select,status-badge,call-button,call-log-form,call
hooks/use-realtime-clinics.ts
middleware.ts
```

Phase 1 | 認証+スキーマ（～半日）

- 上記DDLをSQL Editorで実行
- `middleware.ts` でセッション更新、未ログインは `/login` へ
- メンバーをダッシュボードから招待 → 各自 `profiles` 作成（初回ログイン時にupsert）

Phase 2 | 一覧表示+ワンタップ発信（1日）★ここで「使える」最初の形

- `/clinics` をServer Componentで `select * from clinics` 取得
- `ClinicCard` 描画、`CallButton` で `tel:` 発信
- この時点でスマホから「医院を見てタップ発信」が成立

Phase 3 | ステータス変更+メモ登録（1日）

- `StatusSelect` で楽観的更新
- 詳細画面に `CallLogForm`（結果+メモ→ `call_logs` にinsert、同時に `clinics.latest_memo / status` を更新）
- `CallLogTimeline` で履歴表示

Phase 4 | リアルタイム同期（～半日）★要件4の達成

- `useRealtimeClinics` を一覧に組み込み
- 2端末で開いて、片方の更新がもう片方に即反映されるか検証

Phase 5 | 検索・絞り込み・グループ表示・担当割り当て（～1日）

- `FilterBar`（医院名検索+都道府県／市区町村／ステータス絞り込み。市区町村は都道府県に連動）
- `GroupToggle` + `GrouppedClinicList`（都道府県別・市区町村別・ステータス別にセクション分割、件数バッジ付き）
- 絞り込み・グループ化は §4 の派生ロジックで実装（サーバー再取得なし）

- `assigned_to` の設定UI

Phase 6 | データ投入+デプロイ（～半日）

- データ投入（MVPの現実解）： Googleマップで対象医院を収集 → スプレッドシートでCSV化 → Supabaseダッシュボードの **CSVインポート** で `clinics` に一括投入。Google Places API の自動連携はv2で。
 - 都道府県／市区町村の分割： 絞り込み・グループ化を効かせるため、住所文字列を `prefecture` ・ `city` 列に分けて投入する。日本の住所は「都/道/府/県」で都道府県、続く「市/区/町/村」で市区町村を切り出せる（スプレッドシートの数式か、 `japanese-address-parser` 等のライブラリで一括処理）。手動収集の段階で列を分けておくのが最も楽。
 - 診療時間： 表示用の `business_hours`（テキスト）はGoogleマップの記載をそのまま貼ればOK。「営業中のみ表示」を効かせたい場合のみ `hours`（JSONB）も埋める。Places APIを使うなら `opening_hours.weekday_text` → `business_hours`、`opening_hours.periods` → `hours` にほぼそのまま変換できる。手動なら当面 `hours` は空（`{}`）でよく、その医院は営業中バッジ・絞り込みの対象外になるだけ（テキストは表示される）。
 - ※ Places由来データの保存・キャッシュには利用規約上の制約があるため、初期は手動収集が安全。
- VercelにGitHub連携でデプロイ、環境変数を設定

MVPのゴール状態： 「メンバーがスマホでログイン → 一覧で医院を見る → タップ発信 → 結果とメモを登録 → 全員の画面に即反映」。これが Phase 4 完了時点で揃う。最短で**実働4～6日**規模。

付録：将来拡張（MVP外）

- Google Places API連携での医院データ自動取り込み
- 架電予定のリマインド通知（次回 `next_action_at` ベース）
- 担当者別・期間別のKPIダッシュボード（架電数→ヒアリング率→アポ率）
- 本体プロダクト（リコール自動化SaaS）への送客連携：アポ獲得した医院をそのまま顧客候補テーブルへ昇格